Ahmed Irshad Hussain

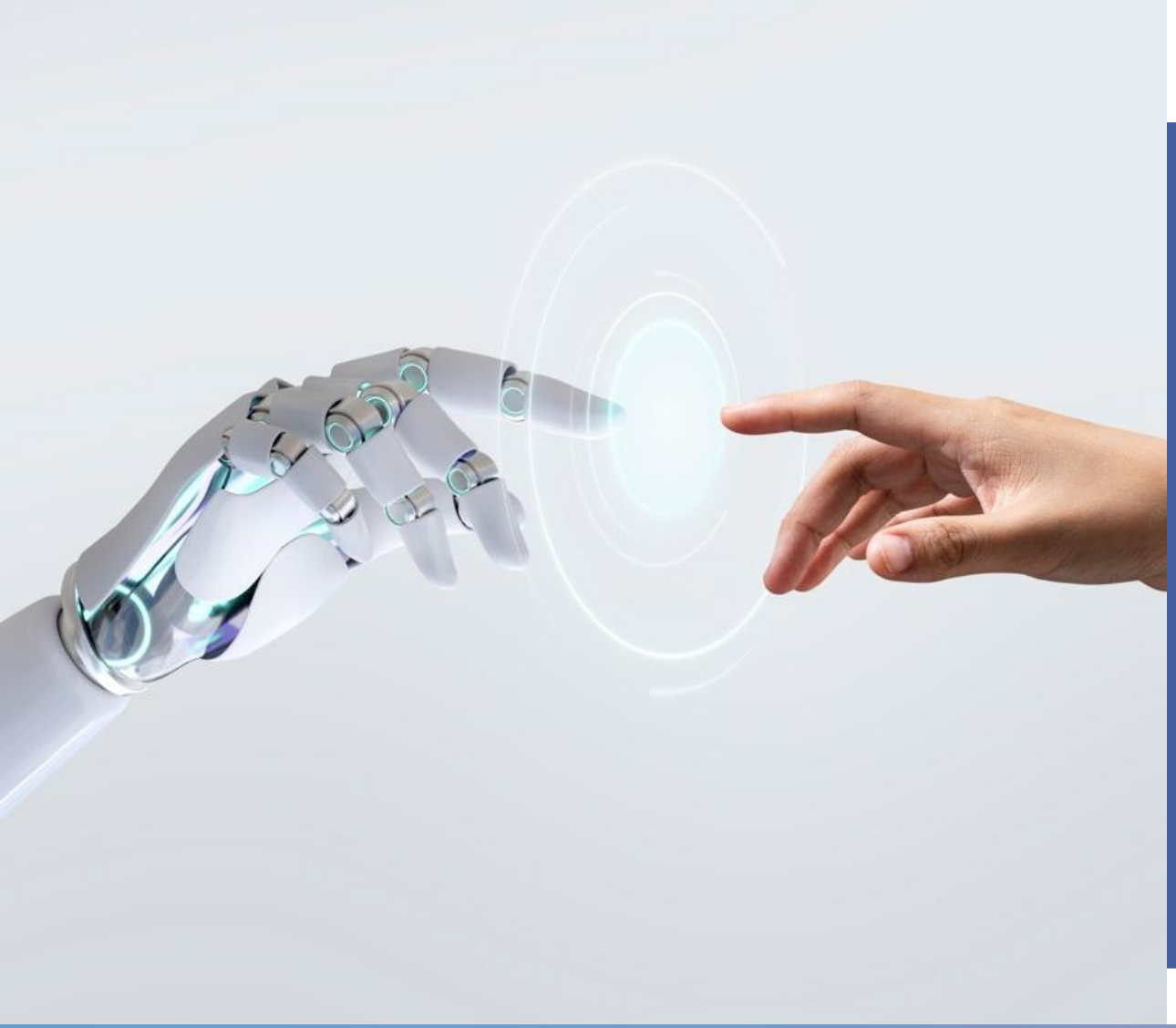
sardarahmed.irshad2@gmail.com

CLOUD DATA ENGINEERING:

MICROSOFT FABRIC

Live API Data Engineering Pipeline

By: Ahmed Irshad Hussain



Data Engineering: Microsoft Fabric
Live API Data Engineering Pipeline

CONCEPT FIRST

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

API Ingestion

This is one of the MOST important skills for modern data engineers.

Because in industry:

data rarely comes from CSVs,

systems continuously generate live data through:

- APIs
- applications
- web services
- IoT devices
- streaming systems

WHAT IS AN API?

API = Application Programming Interface

Think of it as:

A way for systems to communicate and exchange data

REAL-WORLD EXAMPLES

Companies pull data from APIs like:

- weather APIs
- payment APIs
- e-commerce APIs
- social media APIs
- stock market APIs
- healthcare systems

WHAT DATA ENGINEERS DO

Data Engineers:

- Call APIs
- Extract JSON data
- Store raw data in Bronze
- Clean in Silver
- Aggregate in Gold
- Visualize in Power BI

This is REAL enterprise ingestion.

MY PROJECT EVALUATION

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

Current:

- CSV → Bronze → Silver → Gold

After this:

- API → Bronze → Silver → Gold

Now my pipeline will become:

- dynamic and live

WHY APIs MATTER

Because APIs teach:

- live ingestion
- automation
- JSON handling
- schema evolution
- production pipelines

These are CORE modern data engineering skills.

EXPECTED KEY LEARNINGS:

- REST APIs
- JSON Parsing
- API Authentication
- Dynamic Bronze Ingestion

MY PIPELINE WILL BECOME

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

API Source
↓
Bronze (Raw JSON)
↓
Silver (Cleaned)
↓
Gold (Business Layer)
↓
Power BI

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

Microsoft Fabric

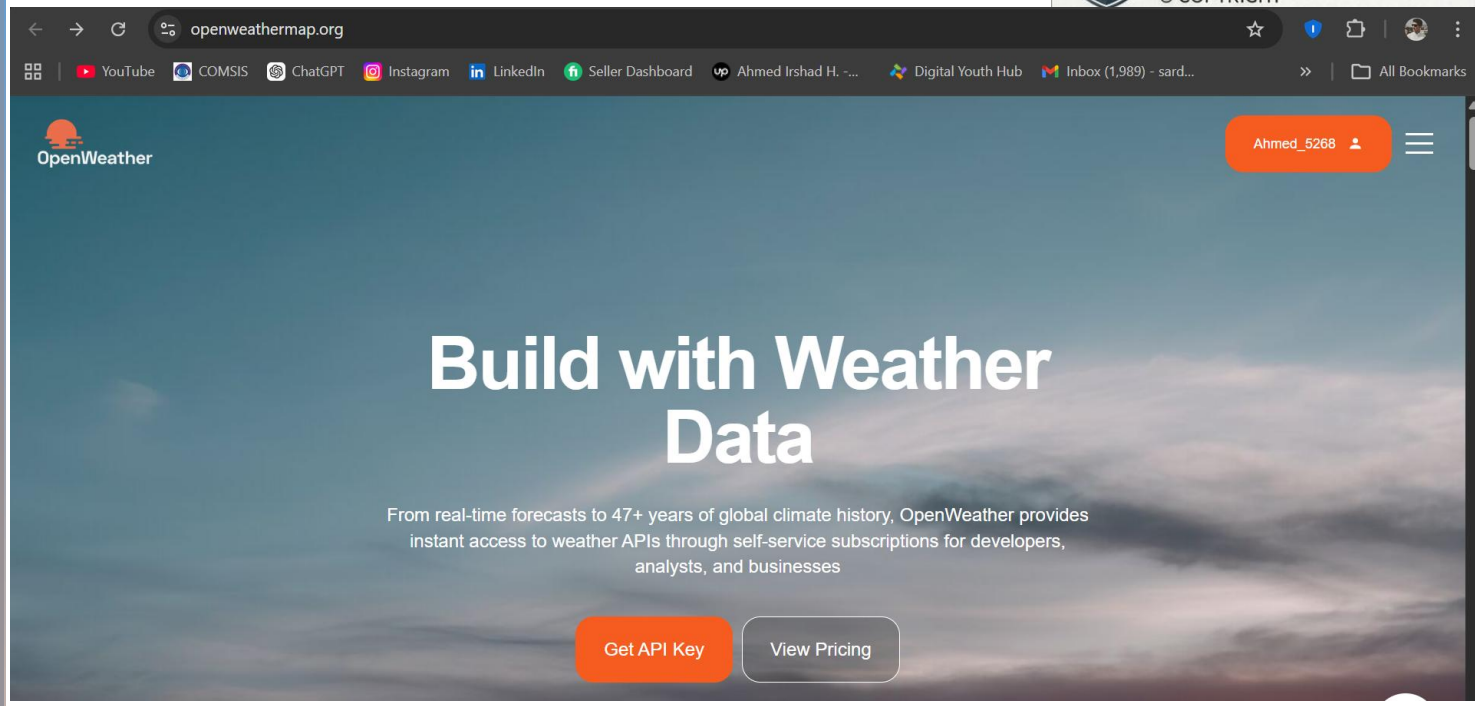
Weather API

I'll use weather API for this project

STEP 1: Go to "Open Weather"

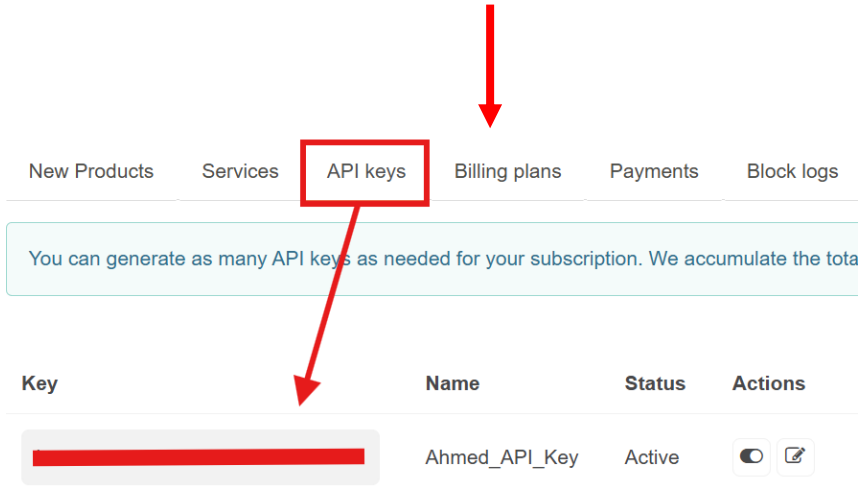
- Create your account first.
- After that click your user's name tab!

  (You can see it at top right corner)



STEP 2: Open API Tab:

- Click API keys
- And generate your own API key.



Microsoft Fabric

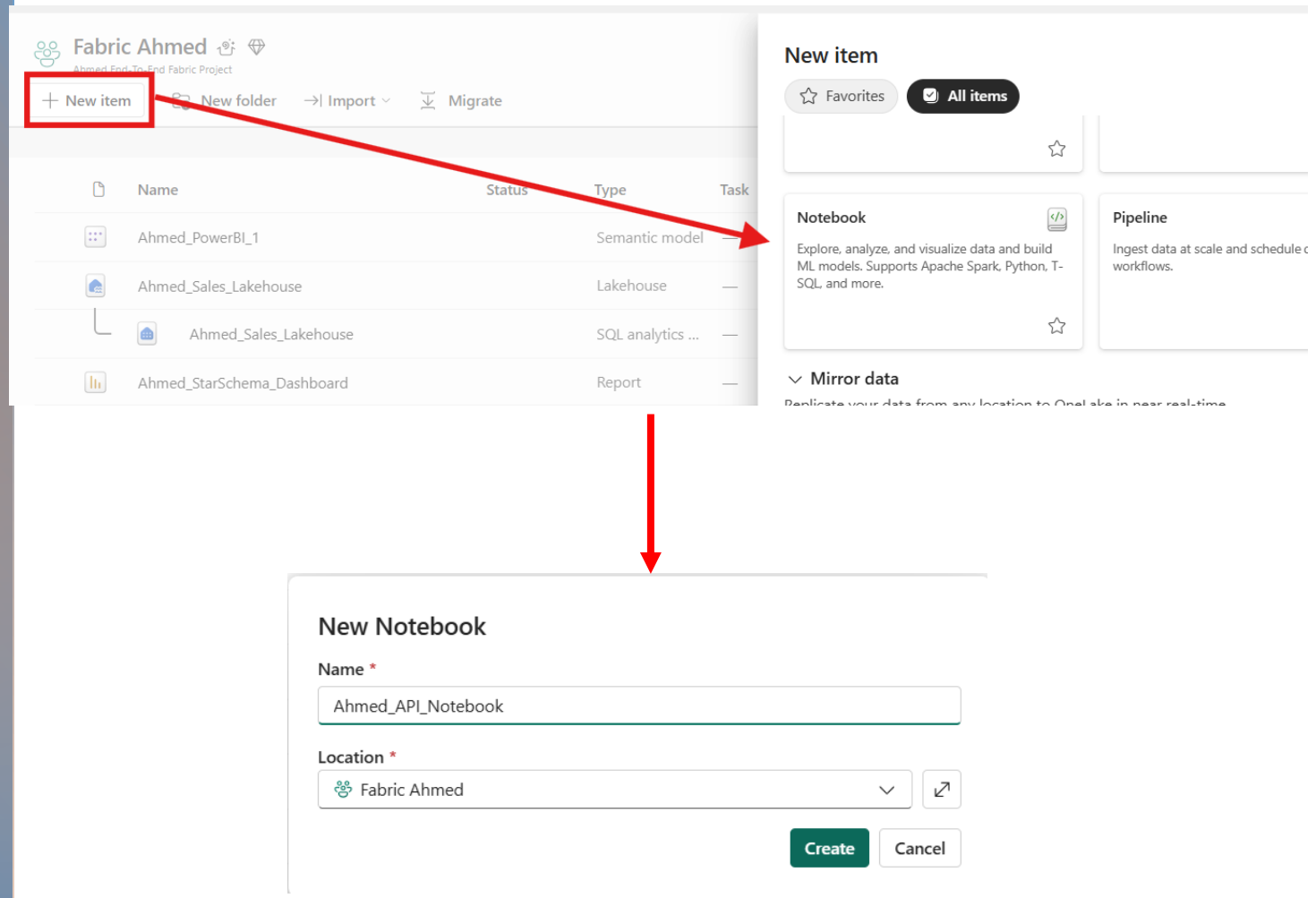
Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

STEP 3: Creating New Notebook

- I'll create new notebook for this project.

  As my old notebook was based on CSV files.
- Naming it "Ahmed API Notebook"

  You all can choose what you like!



Microsoft Fabric

STEP 4: API Ingestion

- Inside code, write first API call!

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com


```
1   import requests
2   import json
3
4   API_KEY = "[redacted]"
5
6   url = f"https://api.openweathermap.org/data/2.5/weather?q=London&appid={API_KEY}&units=metric"
7
8   response = requests.get(url)
9
10  print("Status Code:", response.status_code)
11
12  data = response.json()
13
14  print(json.dumps(data, indent=4))
15
```

Requests = Used to call APIs over internet
JSON= Used to format and work with JSON data

API_Key will store my key for authentication credential.
Without it, API rejects request

This builds the API endpoint dynamically.
q= London, means CITY
appid = API_Key, for authentication
Units = metric, means Celsius Temp.

These 2 lines, will send HTTP request.
And as well as, will check response status.

API returns raw JSON, it'll convert into python dictionary format

This line explanation is in next slide!

[1] ✓ 11 sec - Session ready in 11 sec 178 ms. Command executed in 533 ms by Ahmed Fabric on 5/11/2026, 12:07:50 AM

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com


```
Status Code: 200
{
    "coord": {
        "lon": -0.1257,
        "lat": 51.5085
    },
    "weather": [
        {
            "id": 804,
            "main": "Clouds",
            "description": "overcast clouds",
            "icon": "04d"
        }
    ],
    "base": "stations",
    "main": {
        "temp": 11.68,
        "feels_like": 10.39,
        "temp_min": 10.95,
        "temp_max": 12.38,
        "pressure": 1014,
        "humidity": 57,
        "sea_level": 1014,
        "grnd_level": 1009
    },
    "visibility": 10000,
    "wind": {
        "speed": 4.63,
        "deg": 40
    },
    "clouds": {
        "all": 100
    },
    "dt": 1778439324,
    "sys": {
        "type": 2,
        "id": 268730,
        "country": "GB",
        "sunrise": 1778386604,
        "sunset": 1778441828
    },
    "timezone": 3600,
    "id": 2643743,
    "name": "London",
    "cod": 200
}
```

As, status code = 200
It means, response successfully generated

Last line was for “Pretty Print JSON”

Example:
Without it:
{'temp':20,'humidity':60}

With It:

```
{
   "temp": 20,
   "humidity": 60
}
```

Much Cleaner, as you can see in success status response.

Microsoft Fabric

STEP 5: Connecting Notebook to lakehouse

- Click, Add new items.
- Select “One Lake Catalog”.
- Then add the lakehouse.

STEP 6: Save into Bronze

Currently:

- data exists only temporarily in memory

I’ll now persist/store it into:

- Delta Table, inside my Lakehouse.

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

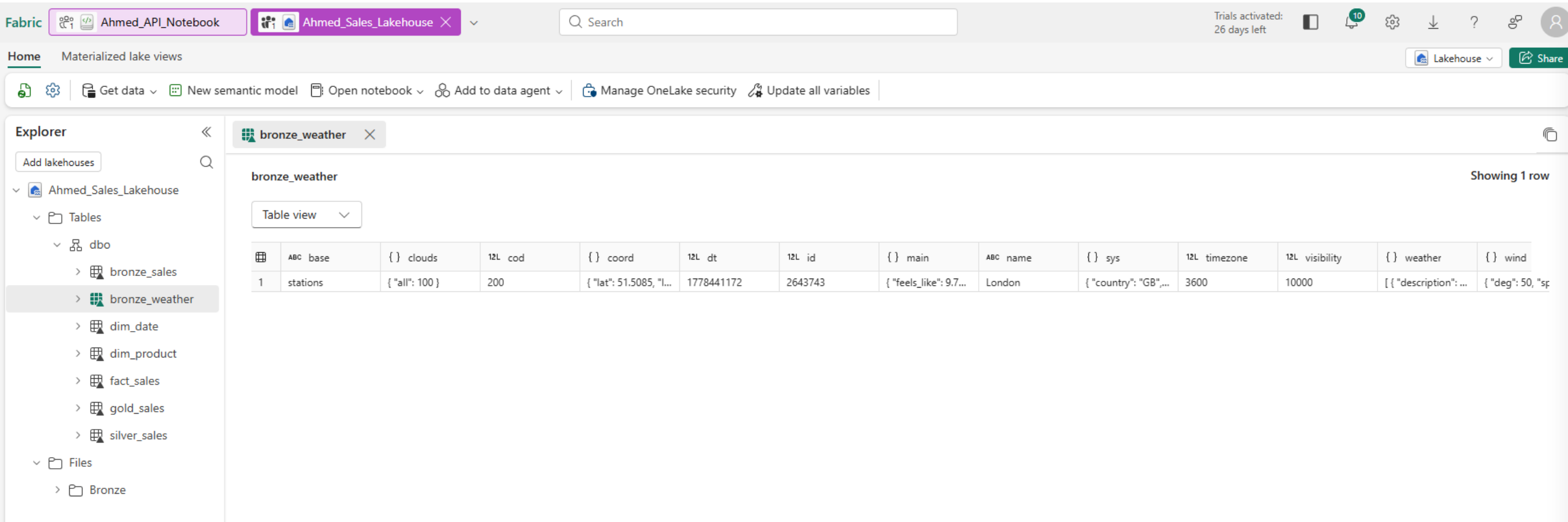

```python
import requests
import json

# API Key
API_KEY = "ba979c51e5a6d44c250e53ab25c603bb"

# API URL
url = f"https://api.openweathermap.org/data/2.5/weather?q=London&appid={API_KEY}&units=metric"

# Fetch Data
response = requests.get(url)

# Convert response to JSON
data = response.json()

# Convert JSON dictionary to string
json_data = json.dumps(data)

# Create Spark DataFrame
df = spark.read.json(sc.parallelize([json_data]))

# Show DataFrame
df.show()

# Save Bronze Layer
df.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("bronze_weather")
```

[2] ✓ 24 sec - Command executed in 24 sec 316 ms by Ahmed Fabric on 5/11/2026, 12:34:37 AM

Spark jobs (11 of 11 succeeded) Resources

VERIFYING

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com



Microsoft Fabric

SILVER LAYER

Currently Bronze contains:

- nested JSON
- raw structure
- unnecessary metadata

Silver layer will:

- flatten JSON
- clean columns
- select useful fields
- create structured analytics-ready table

STEP 7: Create the Silver Layer

- Click, Add new items.
- Select “One Lake Catalog”.
- Then add the lakehouse.

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com


```python
from pyspark.sql.functions import col

# Read Bronze Layer
df = spark.table("bronze_weather")

# Flatten & Clean Data
silver_df = df.select(
    col("name").alias("city"),
    col("main.temp").alias("temperature"),
    col("main.humidity").alias("humidity"),
    col("weather")[0]["main"].alias("weather_condition"),
    col("wind.speed").alias("wind_speed"),
    col("sys.country").alias("country")
)

# Show Silver Data
silver_df.show()

# Save Silver Layer
silver_df.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("silver_weather")
```

[3] ✓ 37 sec - Command executed in 37 sec 625 ms by Ahmed Fabric on 5/15/2026, 5:09:39 PM

What this does?

Bronze (Raw JSON)
Messy nested structure:

	base	clouds	cod	coord	dt	id	main	name
1	stations	{ "all": 75 }	200	{ "lat": 51.5085, "...	1778845875	2643743	{ "feels_like": 10....	London

Silver Structured:

	city	temperature	humidity	weather_condition	wind_speed	country
1	London	12.12	58	Clouds	5.66	GB

Microsoft Fabric

GOLD LAYER

We'll create:

- temperature categories
- weather status
- analytics-ready fields

Example:

city	temperature	temp_category
London	12	Cold

```
1  from pyspark.sql.functions import when, col
2
3  # Read Silver Layer
4  df = spark.table("silver_weather")
5
6  # Create Gold Layer
7  gold_df = df.withColumn(
8      "temp_category",
9      when(col("temperature") < 10, "Cold")
10     .when(col("temperature") < 20, "Moderate")
11     .otherwise("Hot")
12 )
13
14 # Show Gold Data
15 gold_df.show()
16
17 # Save Gold Layer
18 gold_df.write.format("delta") \
19     .mode("overwrite") \
20     .saveAsTable("gold_weather")
```

[4] ✓ 14 sec - Command executed in 14 sec 71 ms by Ahmed Fabric on 5/15/2026, 5:26:21 PM


```
+------+-----------+--------+----------------+----------+-------+-------------+
|  city|temperature|humidity|weather_condition|wind_speed|country|temp_category|
+------+-----------+--------+----------------+----------+-------+-------------+
|London|      12.12|      58|          Clouds|      5.66|     GB|     Moderate|
+------+-----------+--------+----------------+----------+-------+-------------+
```

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

NEXT

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

WHAT I JUST BUILT

I now have:

- LIVE API → Bronze → Silver → Gold , pipeline.

What's Next?

NOW I'll evolve this into: ENTERPRISE API ENGINEERING

Because currently: I fetch one snapshot only

Right now: London → current weather gets overwritten every run.

Instead: I'll APPEND records continuously.

Meaning:

city	temp	ingestion_time
London	12	5:00 PM
London	13	6:00 PM
London	11	7:00 PM

NOW: trends become possible , analytics become meaningful and time-series engineering begins

WHAT I'LL IMPLEMENT NEXT

- ingestion timestamp
- append mode
- incremental API ingestion
- partitioning by date
- historical analytics
- automated scheduling

THIS becomes:

production-grade API pipeline

TIME-SERIES ENGINEERING + LIVE INGESTION SYSTEMS

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

Now I'll move from "Current weather Snapshot" to "Historical weather pipeline".

This is incremental load, but now for APIs!

Instead of overwriting:

London → 12°C

Every run will keep history

city	temp	ingestion_time
London	12.1	5:00 PM
London	13.3	6:00 PM
London	11.8	7:00 PM

After this, I can build,

Hourly trends

Time-series analytics

Growth analysis

Historical dashboards

Microsoft Fabric

Step 1 – Add Ingestion Timestamp

Run

```python
import requests
import json
from pyspark.sql.functions import current_timestamp

# API Key
API_KEY = "ba979c51e5a6d44c250e53ab25c603bb"

# API URL
url = f"https://api.openweathermap.org/data/2.5/weather?q=London&appid={API_KEY}&units=metric"

# Fetch API data
response = requests.get(url)

# Convert to JSON
data = response.json()

# Create Spark DataFrame
json_data = json.dumps(data)

df = spark.read.json(sc.parallelize([json_data]))

# Add ingestion timestamp
df = df.withColumn(
    "ingestion_time",
    current_timestamp()
)

# Show result
df.show()
```

[6] ✓ 7 sec - Command executed in 7 sec 962 ms by Ahmed Fabric on 5/23/2026, 2:25:22 PM

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

What's changed,
Earlier:
London | 12°C
Now:
London | 12°C | 2026-05-15 17:40:23
This Timestamp becomes my:
watermark / historical tracking column

Step 2 – Changing Bronze Write Mode:

Replacing: .mode("overwrite")

With: .mode("append")

```python
df.write.format("delta") \
    .mode("append") \
    .saveAsTable("bronze_weather_history")
```

✓ 9 sec - Command executed in 8 sec 401 ms by Ahmed Fabric on 5/23/2026, 2:31:10 PM

Why Append?

Overwrite:

Old data → deleted

New data → replaces it

Append:

Old data stays

New rows added

Microsoft Fabric

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

Step 3 – Historical Silver Layer

Run

```python
from pyspark.sql.functions import col

# Read Bronze History
df = spark.table("bronze_weather_history")

# Flatten JSON + retain timestamp
silver_df = df.select(
    col("name").alias("city"),
    col("main.temp").alias("temperature"),
    col("main.humidity").alias("humidity"),
    col("weather")[0]["main"].alias("weather_condition"),
    col("wind.speed").alias("wind_speed"),
    col("sys.country").alias("country"),
    col("ingestion_time")
)

# Show data
silver_df.show()

# Save Silver history
silver_df.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("silver_weather_history")
```

[8] ✓ 9 sec - Command executed in 9 sec 761 ms by Ahmed Fabric on 5/23/2026, 2:35:43 PM

Spark jobs (13 of 13 succeeded) Resources Log

silver_weather_history

Showing 1 row

Table view

	city	temperature	humidity	weather_condition	wind_speed	country	ingestion_time
1	London	24.13	57	Clouds	2.57	GB	2026-05-23T09:31:04.839Z

Microsoft Fabric

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

Step 4 – Historical Gold Layer

Now, creating business ready historical data

```python
from pyspark.sql.functions import when, col

# Read Silver History
df = spark.table("silver_weather_history")

# Create Gold Layer
gold_df = df.withColumn(
    "temp_category",
    when(col("temperature") < 10,"Cold")
    .when(col("temperature") < 20,"Moderate")
    .otherwise("Hot")
)

# Show data
gold_df.show()

# Save Gold History
gold_df.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_weather_history")
```

[9] ✓ 10 sec - Command executed in 9 sec 915 ms by Ahmed Fabric on 5/23/2026, 2:39:19 PM

Spark jobs (13 of 13 succeeded) Resources

gold_weather_history

Showing 1 row

Table view

	city	temperature	humidity	weather_condition	wind_speed	country	ingestion_time	temp_category
1	London	24.13	57	Clouds	2.57	GB	2026-05-23T09:31:04.839Z	Hot

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

After this, my architecture becomes,

OpenWeather API

↓

bronze_weather_history

↓

silver_weather_history

↓

gold_weather_history

Now each rerun adds a new weather snapshot.

After that, I'll do the next big jump:

- Partitioning by date
- Scheduling automation
- Power BI dashboard using live historical trends
- Incremental API engineering with watermarks

Partitioning For API Historical Data

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

Right now, my gold_weather_history stores all records together.

As history grows:

London → hourly data

1 day → 24 rows

1 month → 720 rows

1 year → 8,760+ rows

multiple cities → much larger

Scanning everything becomes inefficient.

GOAL:

Partition by:

- Year
- month

So, Spark reads only required data.

Example:

Current:

gold_weather_history

└── all data together

After partitioning:

gold_weather_partitioned

```
│
├── year = 2026
│      ├── month = 5
│      ├── month = 6
└── year = 2027
```

Microsoft Fabric

Partition For API Historical Data

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

Step 5 – Historical Gold Layer

Now, creating business ready historical data

```python
from pyspark.sql.functions import year, month, col

# Read Gold History
df = spark.table("gold_weather_history")

# Create partition columns
partitioned_df = df.withColumn(
    "year",
    year(col("ingestion_time"))
).withColumn(
    "month",
    month(col("ingestion_time"))
)

# Show data
partitioned_df.show()

# Save partitioned table
partitioned_df.write.format("delta") \
    .mode("overwrite") \
    .partitionBy("year","month") \
    .saveAsTable("gold_weather_partitioned")
```

[11] ✓ 12 sec - Command executed in 11 sec 980 ms by Ahmed Fabric on 5/23/2026, 3:03:50 PM

Spark jobs (14 of 14 succeeded) Resources Log

What this does?

Adds,

city	temperature	ingestion_time	year	month
London	12.1	2026-05-15	2026	5

Then physically stores data in partitions.

gold_weather_partitioned

Showing 1 row

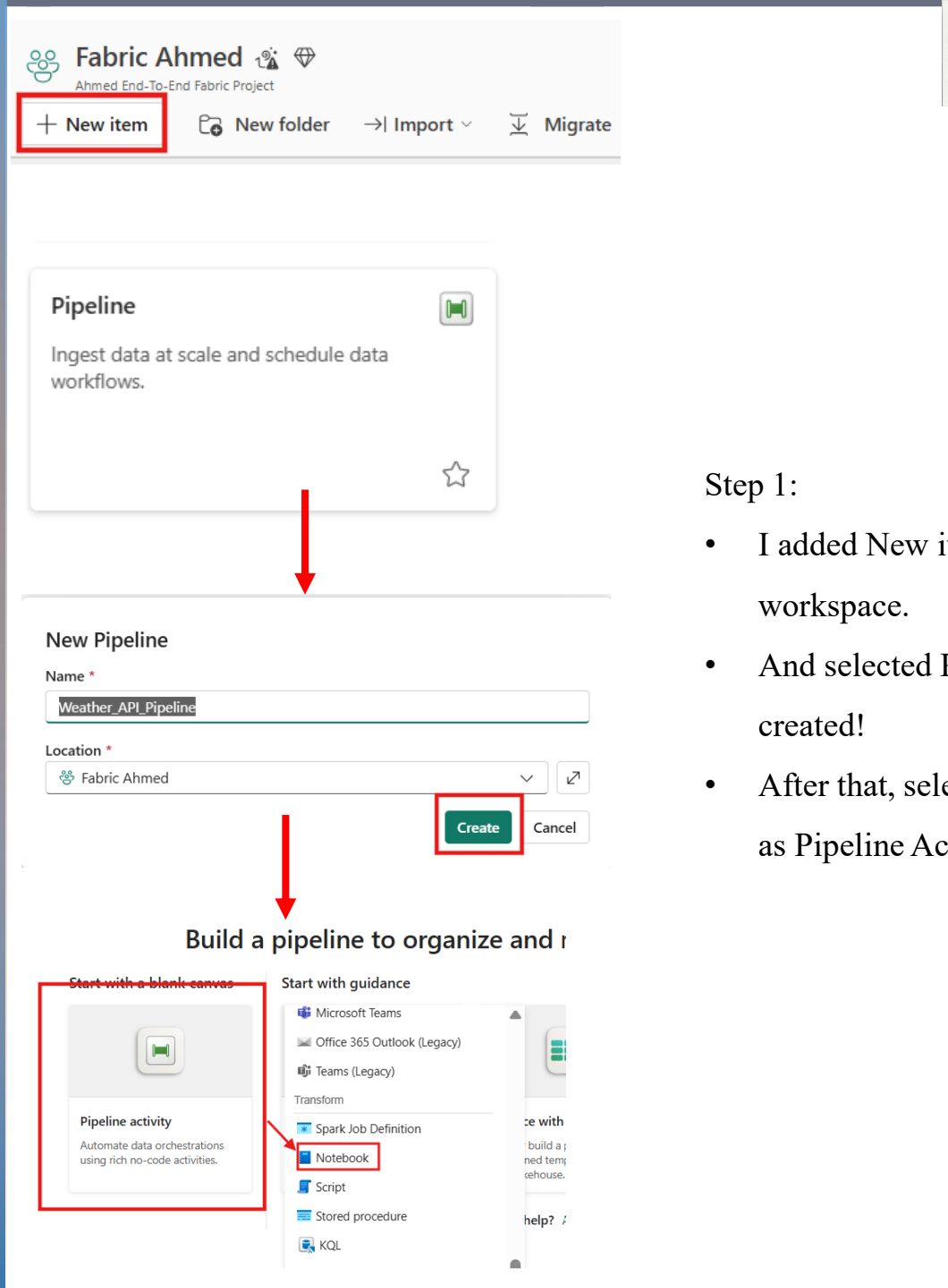
Table view

	city	temperature	humidity	weather_condi...	wind_speed	country	ingestion_time	temp_category	year	month
1	London	24.13	57	Clouds	2.57	GB	2026-05-23T09:31:...	Hot	2026	5

Microsoft Fabric

AUTOMATION (PIPELINE SCHEDULING)

Step 6 – AUTOMATION



Step 1:

- I added New item, inside the workspace.
- And selected Pipeline, then created!
- After that, selected Notebook as Pipeline Activity.

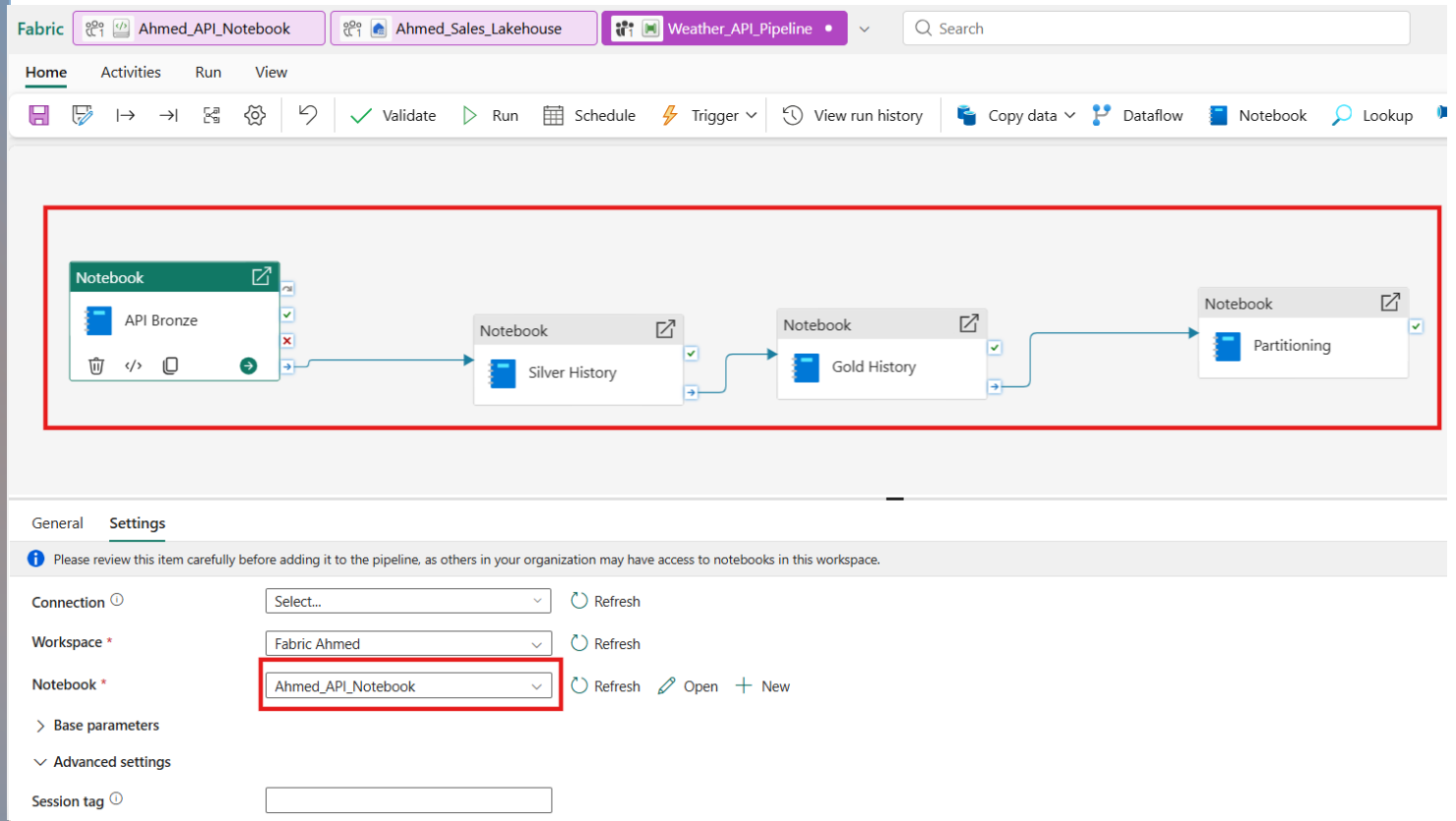
Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

Microsoft Fabric

AUTOMATION (PIPELINE SCHEDULING)

Step 6 – AUTOMATION

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com



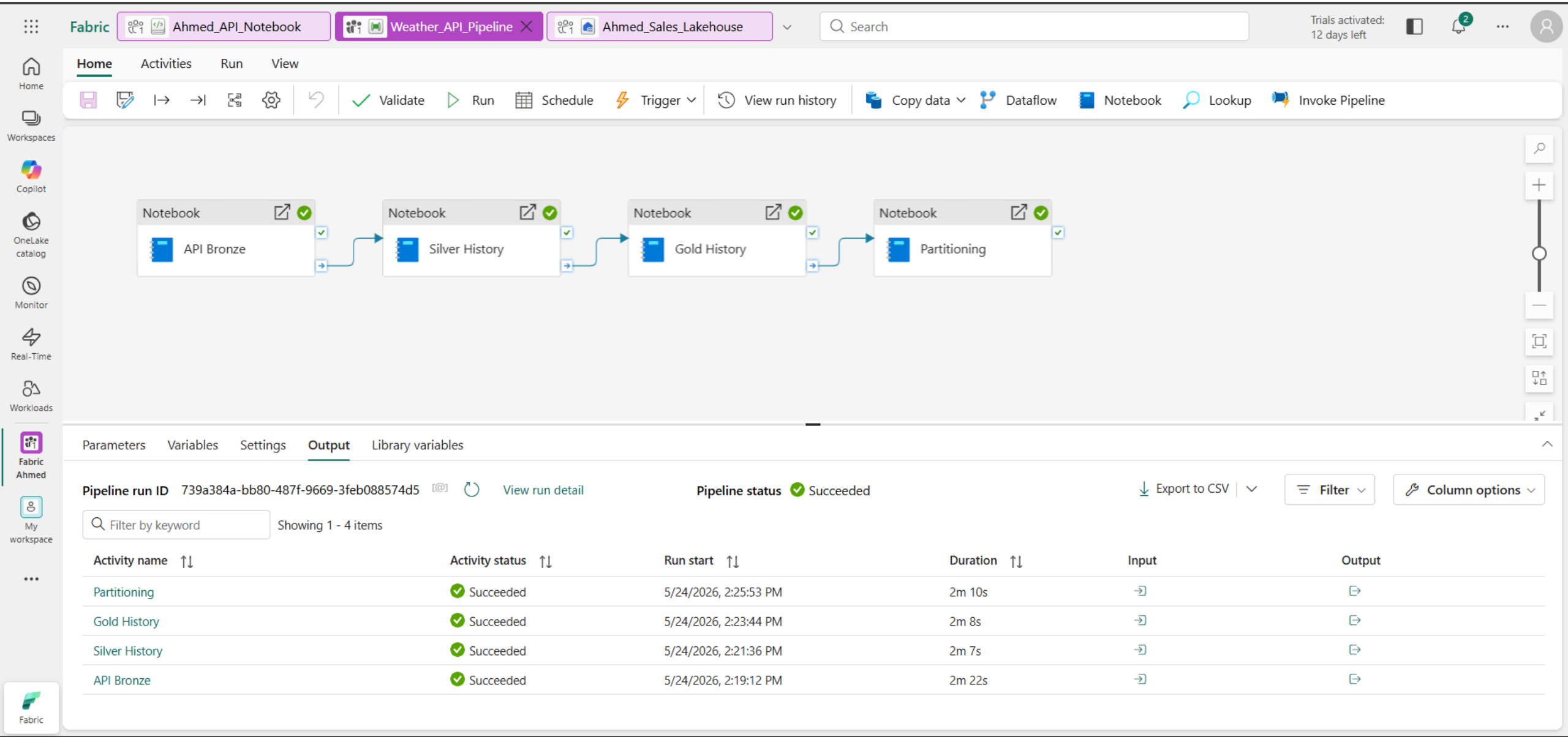
Step 2:

- Inside Pipeline, I added four activities and assigned them the Notebook.
- After that, I connected every activity. (Look at the arrows), it means first bronze will run then silver then gold and then partition.
- After that, select Notebook as Pipeline Activity.

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

Step 3:

- I clicked the Run, to check the status of the pipeline.
- As all succeeded, it means there's no error in my pipeline.



Microsoft Fabric

AUTOMATION (PIPELINE SCHEDULING)

Step 6 – AUTOMATION

Why I set hourly?

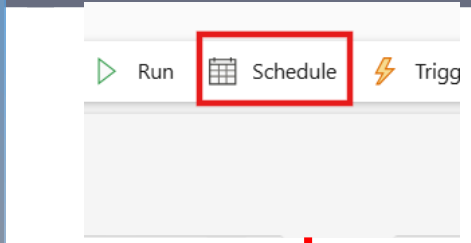
Because weather changes over time:

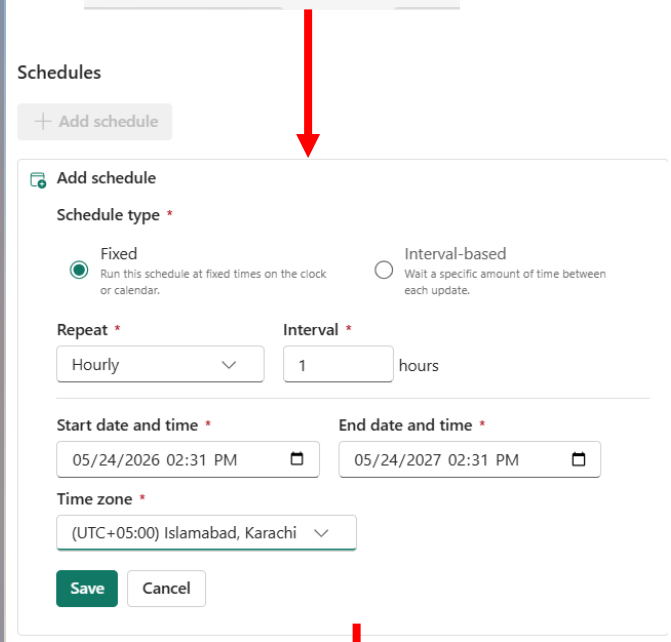
5:00 PM → 12°C

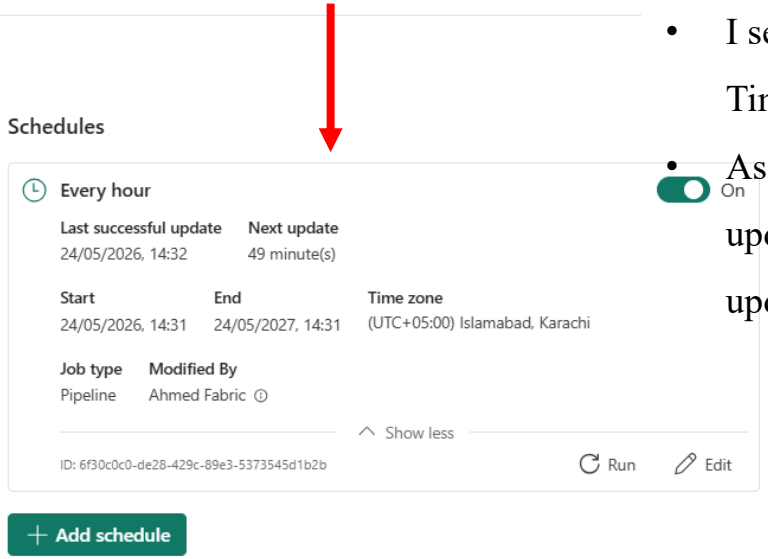
6:00 PM → 13°C

7:00 PM → 11°C

Now history grows automatically.







Step 4 – Add Schedule:

- Click schedule.
- As I used Weather API, so it means I need data every hour.
- I set the schedule hourly based on Pakistani Time!
- As you can see in the third screen shot, Last update was on 2:32 pm (PKT) and the next update will be by 3:32 pm (PKT).

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

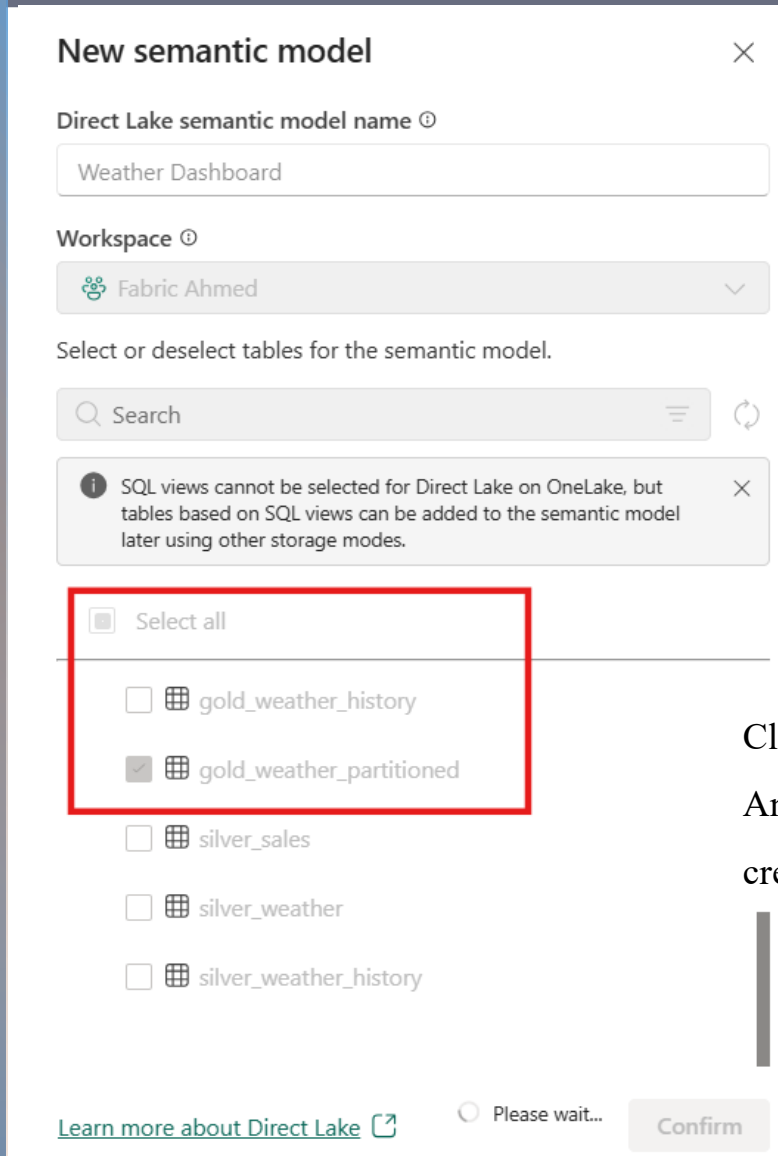
Microsoft Fabric

POWER BI DASHBOARD

Step 7 – POWER BI

I'll use,

- 4 KPI Cards for City, Current Temperature, humidity, Wind Speed &
- a Line Chart for Wind Speed by Ingestion Time)



Click new Semantic Model,

And then choose the table on which you want to create your dashboard!

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

City

London

Current Temperature

Sunny

30.95

Current Temperature KPI

Wind Speed

6.17

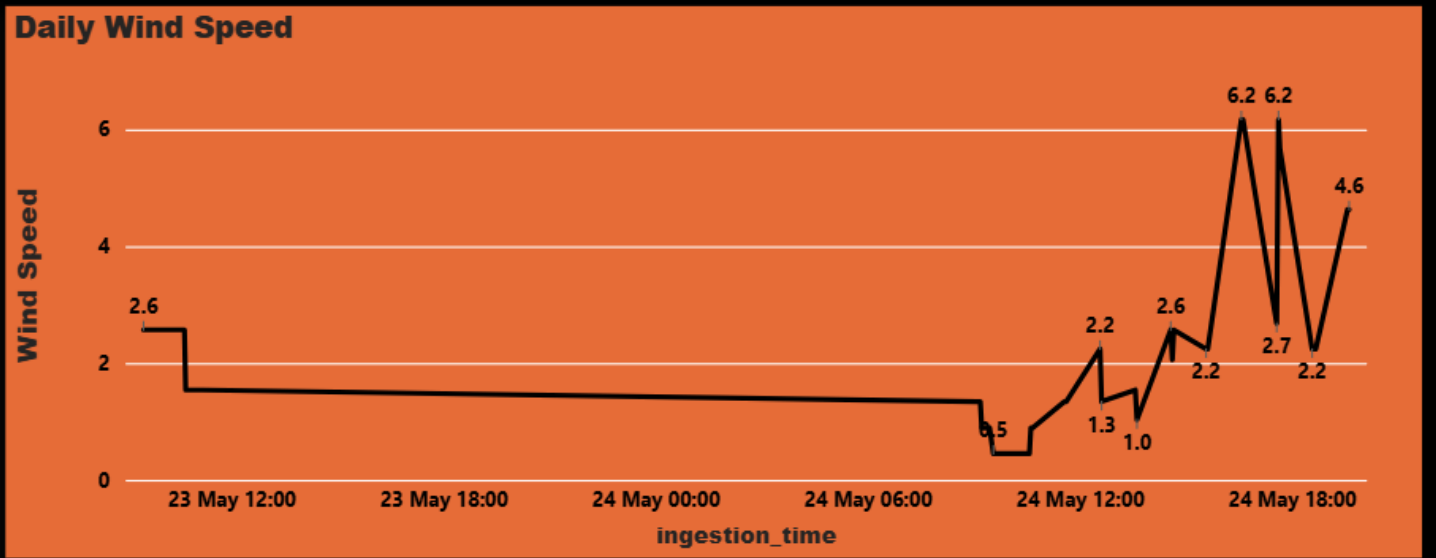
Max Daily Wind Speed

Current Humidity

62

Current Humidity KPI

Daily Wind Speed



Weather API Analytics – End to End Flow

Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com

1. DATA SOURCE

OpenWeather API

OpenWeather

Real-time weather data API

2. INGESTION (BRONZE)

API Bronze

Raw JSON response stored as-is

Table: bronze_weather_history

3. PROCESSING (SILVER)

Silver History

Cleaned, flattened and standardized data

Table: silver_weather_history

4. CURATION (GOLD)

Gold History

Business level data ready for analytics

Table: gold_weather_history

5. OPTIMIZATION

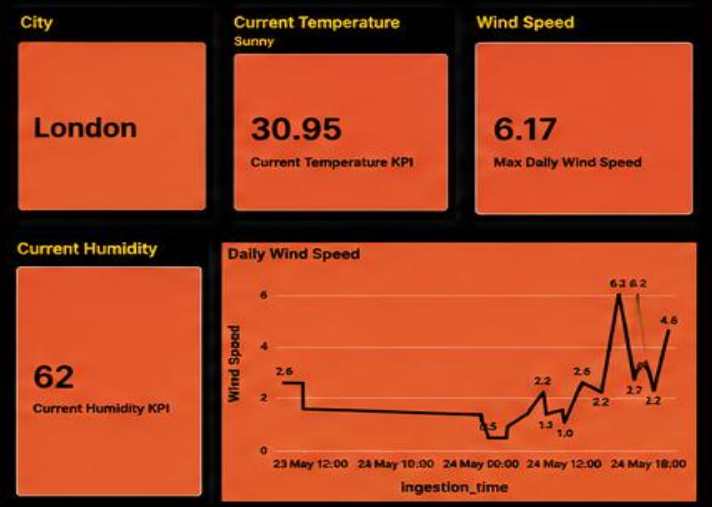
Partitioning

Partitioned by Year and Month

Table: gold_weather_partitioned

6. CONSUMPTION

Weather Dashboard (Power BI)



PIPELINE ORCHESTRATION

Fabric Data Pipeline

API Bronze Notebook → Silver History Notebook → Gold History Notebook → Partitioning Notebook

Sequential Execution
Each step runs after the previous step succeeds

SCHEDULED EXECUTION

Trigger: Recurring
Frequency: Every 1 Hour
(Configurable)

MONITORING

- Monitor Pipeline Runs
- Spark Applications
- Job Concurrency
- Alerts & Notifications

KEY FEATURES

- Automated API data ingestion
- Bronze → Silver → Gold Architecture
- Historical tracking (incremental append)
- Data quality improvement at each layer
- Partitioning for performance optimization
- Orchestration with Fabric Pipeline
- Scheduled execution (Every 1 Hour)
- Interactive Power BI dashboard

TECHNOLOGIES USED

Microsoft Fabric · Notebook (PySpark) · Lakehouse (Delta Tables) · Data Pipeline · Power BI (Dashboard)

BUSINESS VALUE

- Real-time weather visibility
- Historical trend analysis
- Data-driven decision making
- Scalable and automated pipeline
- Ready for enterprise reporting

LEGEND

- Data Flow
- Monitoring / Feedback
- Notebook Activity
- Partitioned Data
- Delta Table